

Asynchronous SAR-ADC Design

Deniz Gorur & Anton Walvoord

Introduction

The SAR ADC (Successive Approximation Register Analog-to-Digital Converter) is an incredibly popular style of ADC that is commonly used for low-power applications due to its straightforward architecture, compelling resolution-to-power tradeoff, and low frequency operation. It functions by using a switching network to toggle a set of binary-weighted capacitors and change the voltage read by a comparator. There are a few different schemes for what voltage and what connection is where, however the general principle is to switch the voltage connected to one capacitor at a time. This will induce a potential change on the other plate of the capacitor, and shift the voltage of the opposite side. This is used to compare against a reference voltage, and as you toggle bits, choosing to keep one set or not, you will sum approximately closer and closer to your voltage given in binary by the comparator result of each comparison. After N conversion cycles, corresponding to the number of bits of resolution of your ADC, a full binary value is found and can be used to calculate the analog voltage read.

There are many different variations of SAR ADC architectures, however, most share the same key fundamental building blocks. There is a sample-and-hold circuit to capture the analog input and hold it steady for the sampling cycle. Next is the Capacitive Digital-to-Analog converter (CDAC), this is the component described earlier that switches the capacitors to produce a different voltage for comparison. Next there is the comparator which evaluates the difference between the held input and CDAC output in each cycle. Finally there is logic to control the switching of values and storage of comparison results. Later in this report we will discuss where SAR ADC architectures diverge.

Design Challenges

There are also a few prominent design challenges that can result in choosing different architectures. One choice is to decide whether to sample on the top plate or bottom plate of your capacitor bank. Sampling via the top plate requires fewer components and control complexity, but also limits your sampling range. .

A very important factor to be mindful of is that since the CDAC is switching capacitive loads, there must be settling time built into our sampling cycle to ensure that the line has had time to stabilize before beginning comparison. Though an 8-bit DAC is not extreme, it is ideally precise down to $\sim 4\text{mV}$ and so the fluctuations in the rail can be significant in determining if a bit is set or not translating into errors near MSB decisions.

Review of Existing Work that Has Been Done to Tackle The Problem

As mentioned prior, SAR ADC's are incredibly popular devices. Because of this there are many implementations to reference from. These devices typically face a tradeoff between power, speed, and size. Generally, SAR ADC's are better suited for applications where low power and small size are the highest priority.

Existing solutions aim to emphasize this strong suit by using dynamic comparators which have incredibly low static power. These comparators, including strong-ARM and double-tail designs, are clocked and pull very little power when inactive. When using a clocked design, the comparator can either be synchronous or asynchronous in order to account for variability in time to evaluate each bit.

Novelty of Our Work and Challenges to Solve

Our work is not particularly novel, however, we sought to piece together techniques we saw in different papers as we compared and contrasted the apparent design trade offs.

We sought to focus on low power, and because of that implemented an asynchronous dynamic comparator design using a double-tail comparator. This minimizes static power and easily allows the user to scale their clock frequency for better performance or power results.

Analysis of design trade off

Through the design of our SAR ADC, we faced many of the challenges described and made several iterations through our design as we adjusted in response to various challenges we faced.

Our initial idea was trying to utilize a top-plate sampling design in an attempt to reduce the complexity of switching components. However, we realized that this ultimately had unintended effects. Our initial implementation failed to account for the required common mode voltage and because of that our circuit had unreliable behavior, working for certain voltage ranges but completely failing for others.

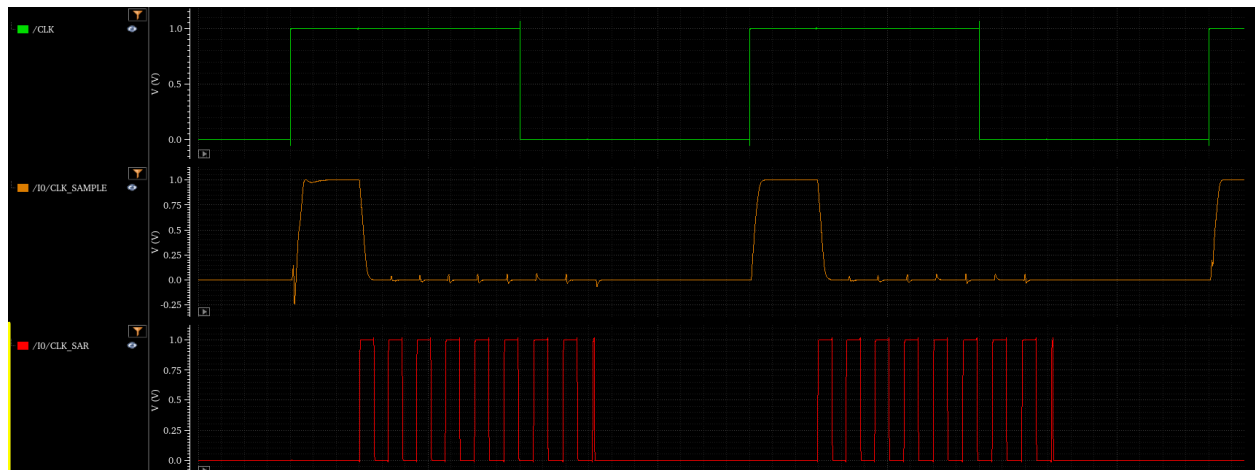
Later, we switched to bottom-plate sampling in an attempt to rectify these issues. While troubleshooting, we discovered that our previous circuit likely wasn't working due to an improperly developed testbench which lacked a common mode voltage input.

Implementation

Our final implementation borrows elements from multiple designs explored during our literature study. Through exploring the various designs of different SAR ADCs, we became most interested in designs that used different techniques to reduce power consumption with a slower frequency and clocked components. Through these techniques, the SAR ADC consumes the most power when sampling, but consumes much less during the rest of operation time.

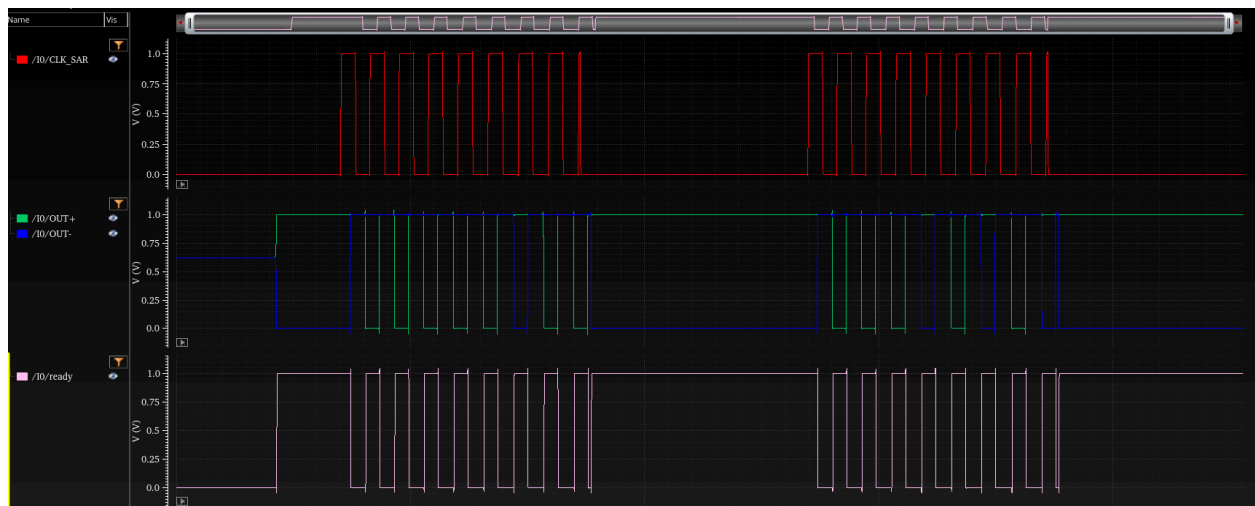
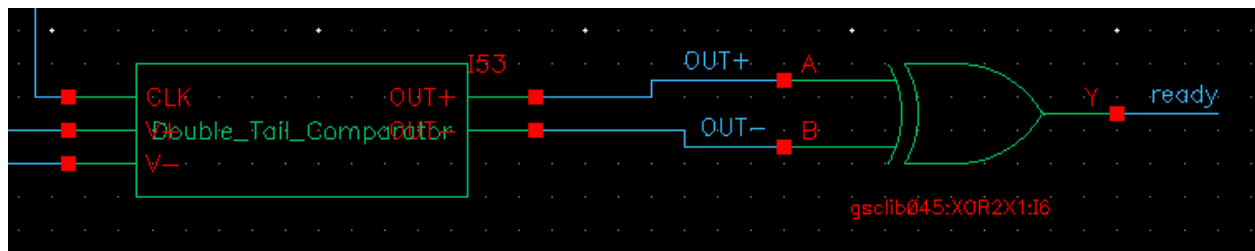
Our largest architectural choice was between a synchronous or asynchronous design. Synchronous ADCs require the usage of a fast external clock that is at the very least equivalent to $f_{\text{sampling}} \times N \text{ bits}$ of resolution as you are making one comparison per clock cycle. Alternatively, asynchronous designs utilize a clock generator that incorporates feedback from the comparator to initiate the next comparison cycle. This way, your external clock is used to set your desired sampling frequency while internally comparison cycles happen as fast as the delays and comparisons take to complete. The usage of a clock generator also helps address a design problem mentioned earlier, which is waiting for the settling of the rail. With a faster clock, the pulse width would have to be sized such that it is slow enough for the rail to settle when the rail is first charged, consequently bounding the remaining bit comparison cycles although there is less voltage fluctuation during them.

Our design implements separate pulses during each sampling cycle. There is 1 pulse with a longer width that we refer to as our sampling clock (or CLK_SAMPLE for short), and a second clock controlled via comparator feedback known as the successive approximation register clock (CLK_SAR for short). Below is a picture of the timing of the pulses relative to our external clock.



The sampling clock will go high long enough to allow the rail to settle when sampling begins, sized via delays. Meanwhile SAR clock takes advantage of the comparator's behavior to generate a feedback mechanism to trigger itself to reset when a bit conversion is complete. When the comparator is not sampling, both outputs: Out+ and Out- remain high. When comparison begins, they both fall until the comparison is complete after which

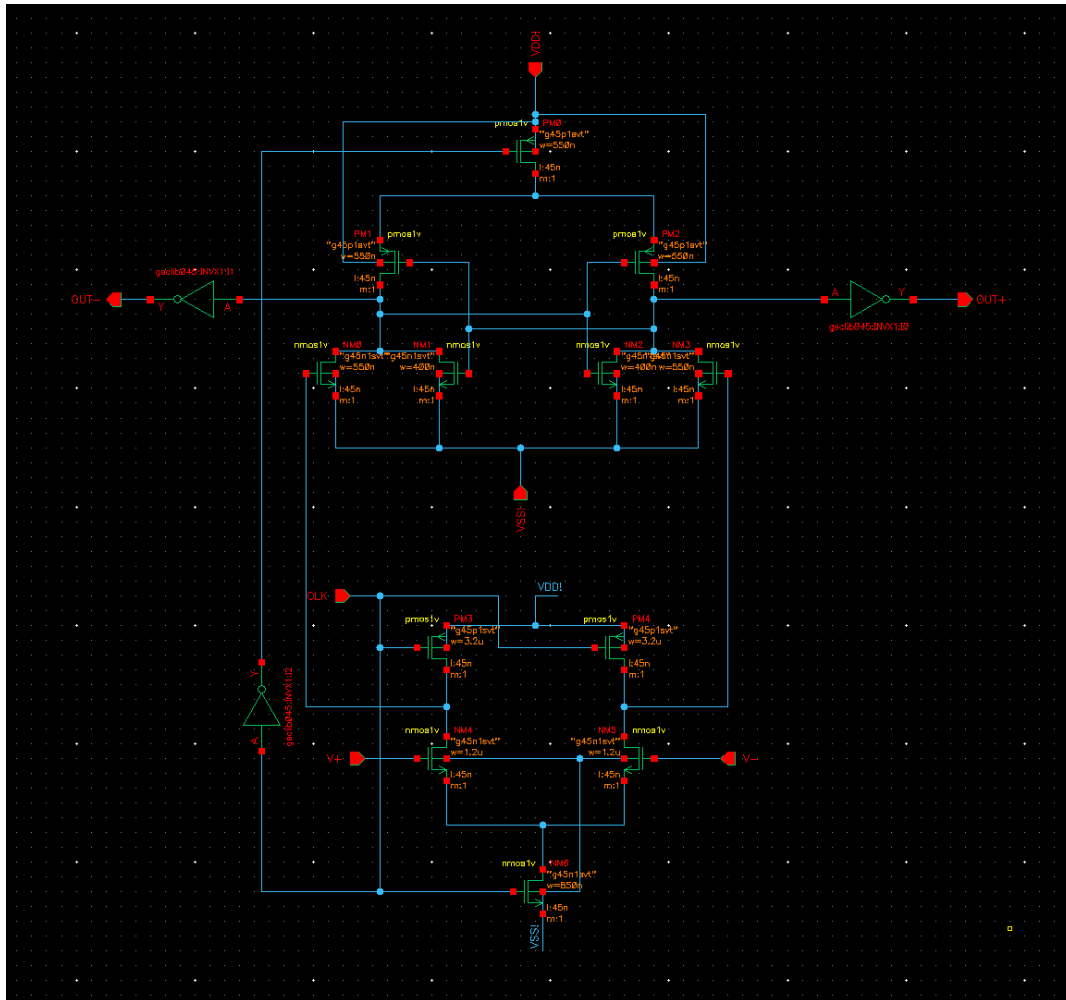
they diverge until one output goes high and the other goes low. By placing a XOR gate on the output of the comparator, we are able to generate a flag that indicates a conversion is complete, and that the comparator is ready for the next bit.



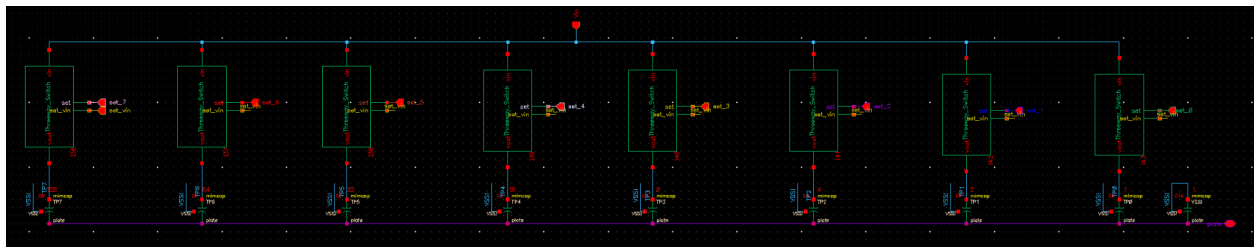
Through this mechanism, the ADC is only converting while it needs to, and can remain idle until it must read the next input. This does result in more complexity in design, dedicating area to a clock generator and the control signals to make it work. Since reducing the frequency is one of our avenues to reduce our power consumption, as dynamic power is described by $\alpha CV^2 f$. Though we did not implement it, if we were to continue tweaking this design in the future we believe a direction would be to implement an error detection mechanism to notify an external digital chip if the conversion is not able to be completed within the external clock's period. There is logic to indicate the end of a conversion and so it can be compared if a new external clock arrives before conversion is complete.

The other key parts of our design were the decisions for our CDAC architecture and comparison approach. While they are independent functional blocks, they require careful balancing of the effects they have on one another in the design of a SAR ADC. Following our motivation to reduce power consumption, we sought to use a dynamic comparator as opposed to something like an op-amp as it would consume less energy, as well as be smaller in area.

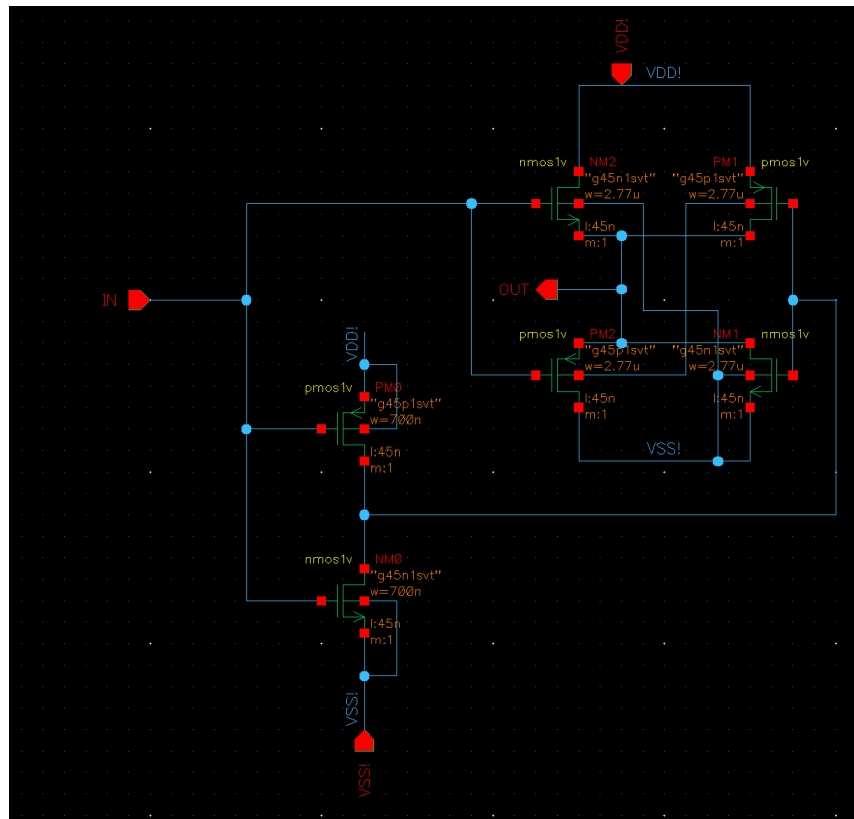
Our main decision of dynamic comparator was between a strongarm and a doubletail comparator due to the lower power consumption compared to utilizing something such as an op-amp. We originally iterated through a strongarm implementation but were unable to produce accurate results at low voltages, heavily biasing towards the negative rail as the latch was unable to regenerate. Through further testing, we opted for the double tail comparator and utilized a command-mode input V_{cm} of $(V_{dd}/2) = 0.5V$ as the reference input.

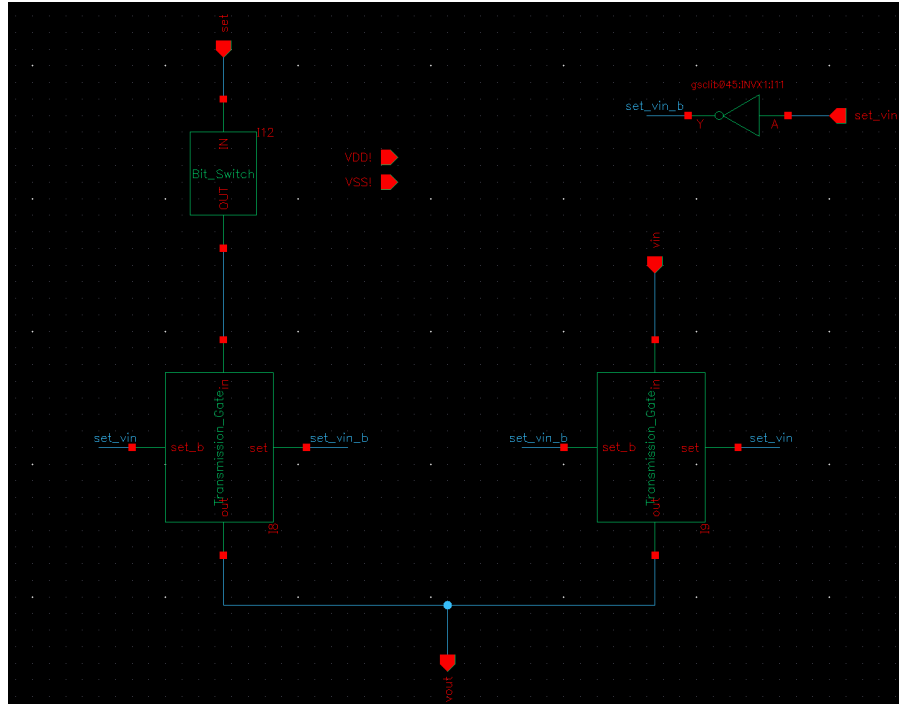


With the use of a common mode voltage, we implemented bottom plate sampling in the CDAC as follows:

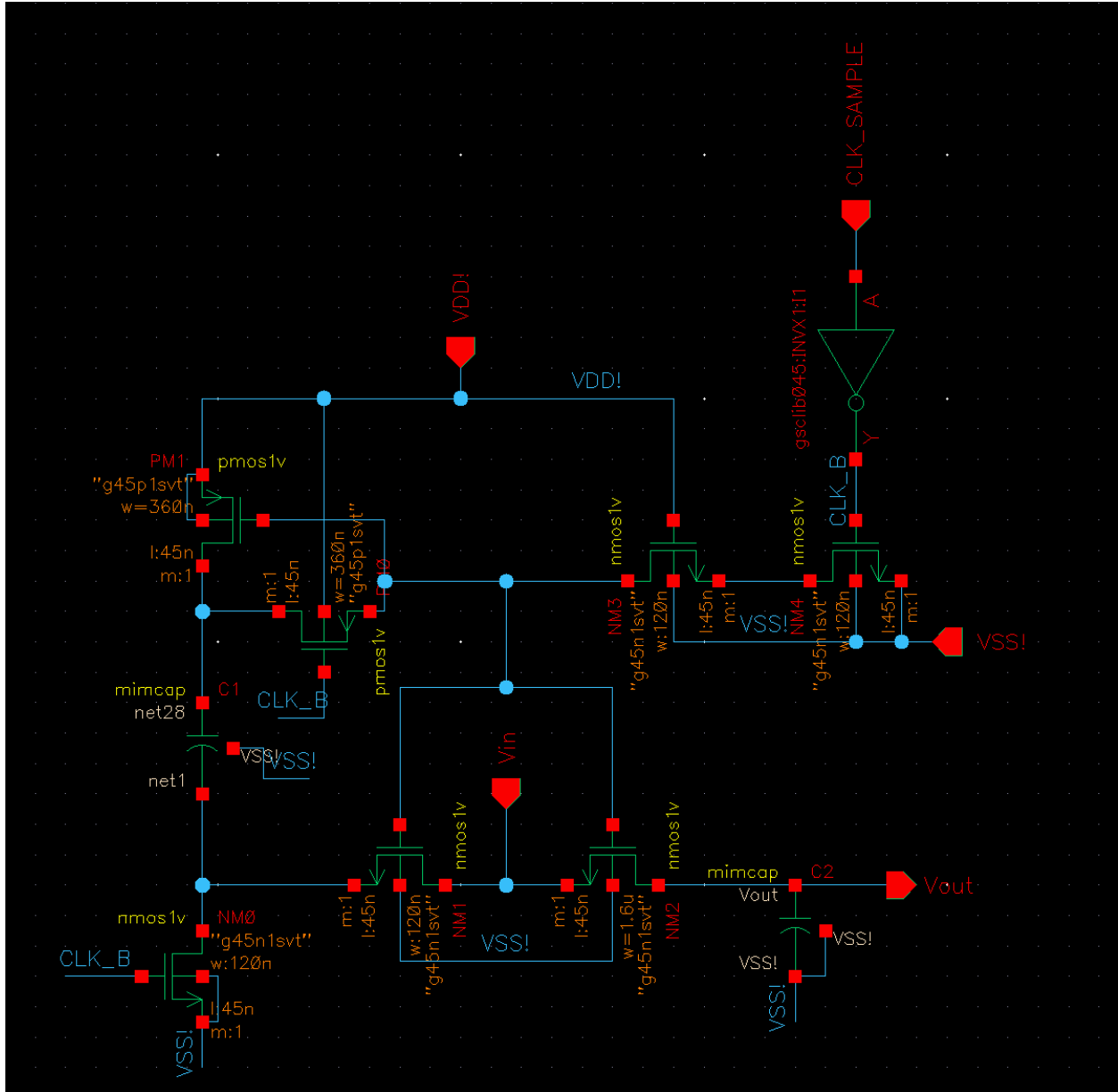


Below is our design for the bit switch that feeds a high or low signal into the three-way switch along with CMOS transmission gates.

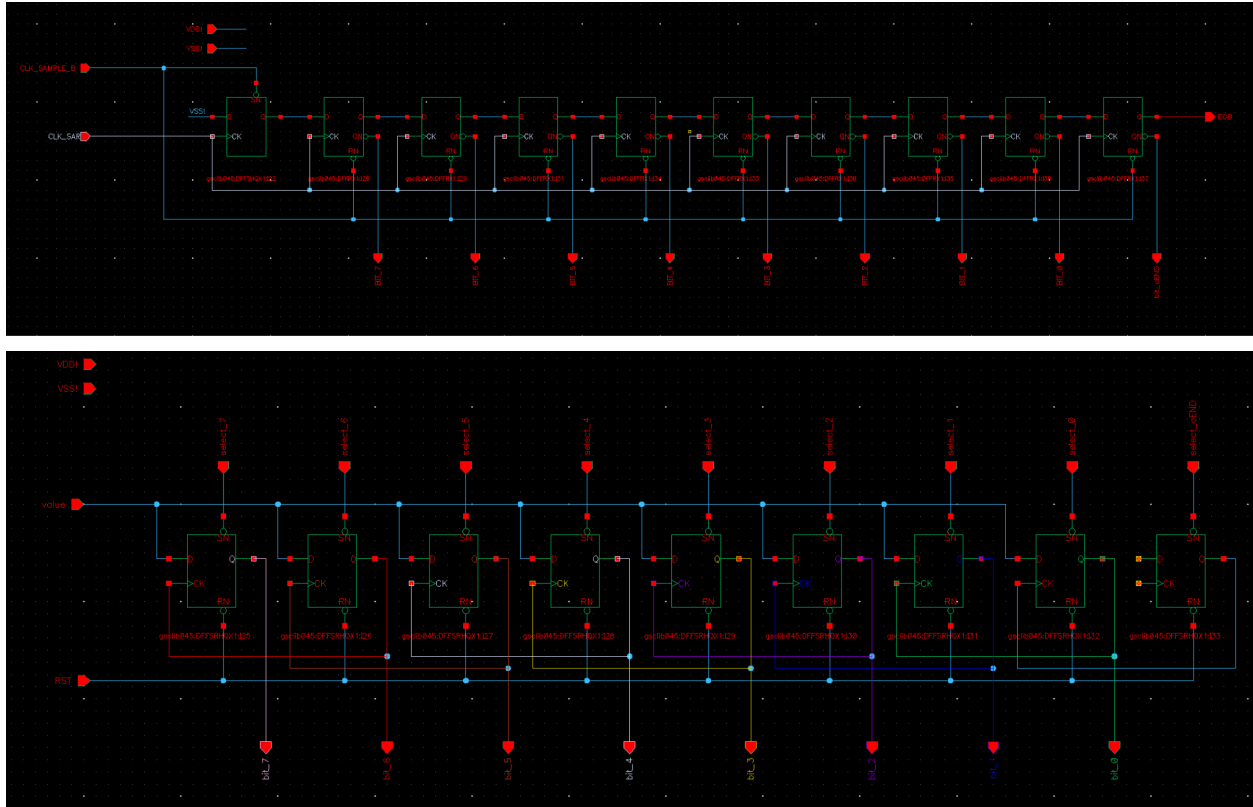




Below is our sample and hold circuit. We opted for a bootstrapped switch to maintain a consistent $R_{ds\ on}$. This way, while there are losses, there is less variation across the range of voltages and this is something that can be mitigated via a fixed offset from the external results.

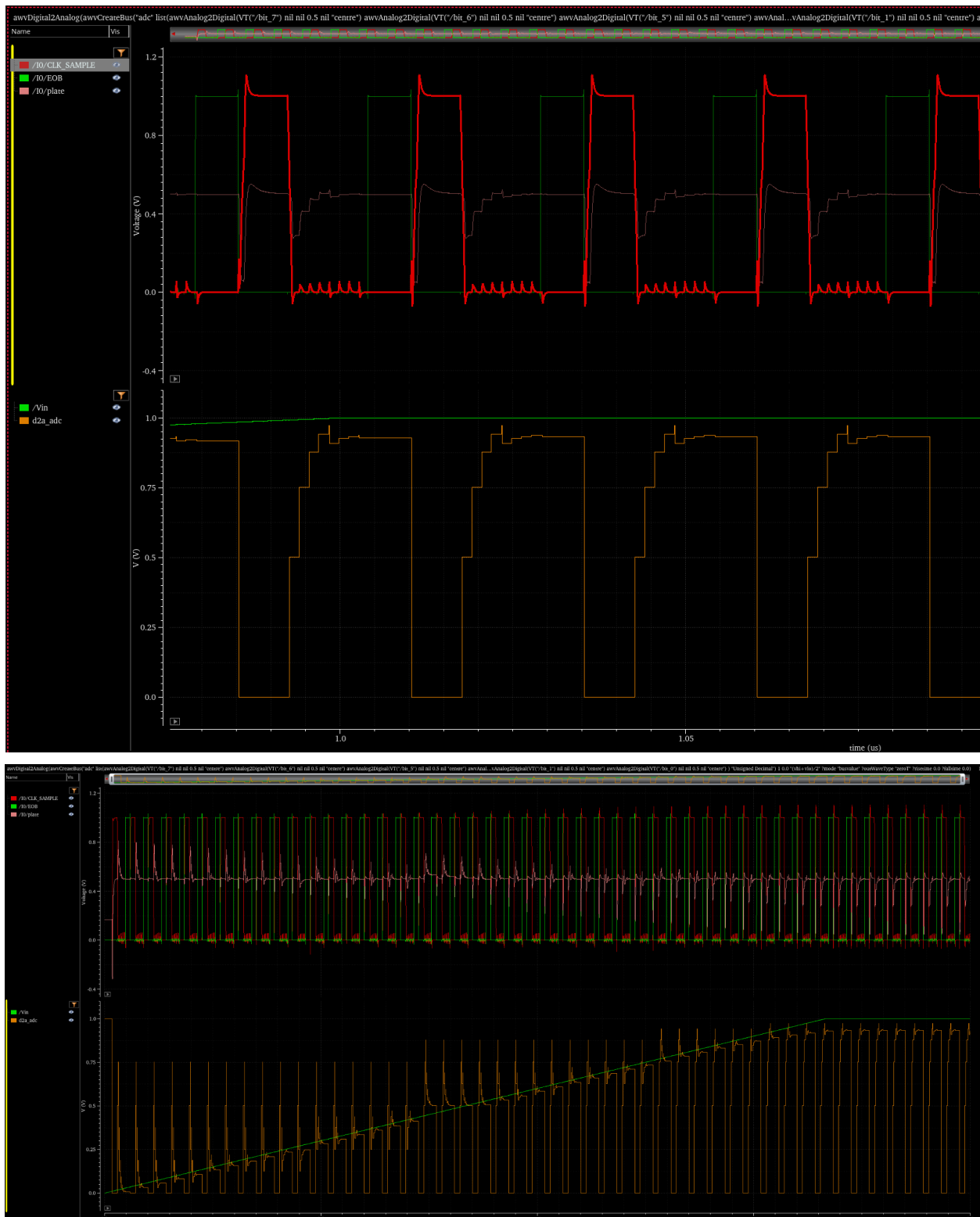


Finally, the data storage is set up such that we are able to do our selection of which bit we are sampling and the storage of the bits we keep only with flips-flops and switches.



Simulation Results

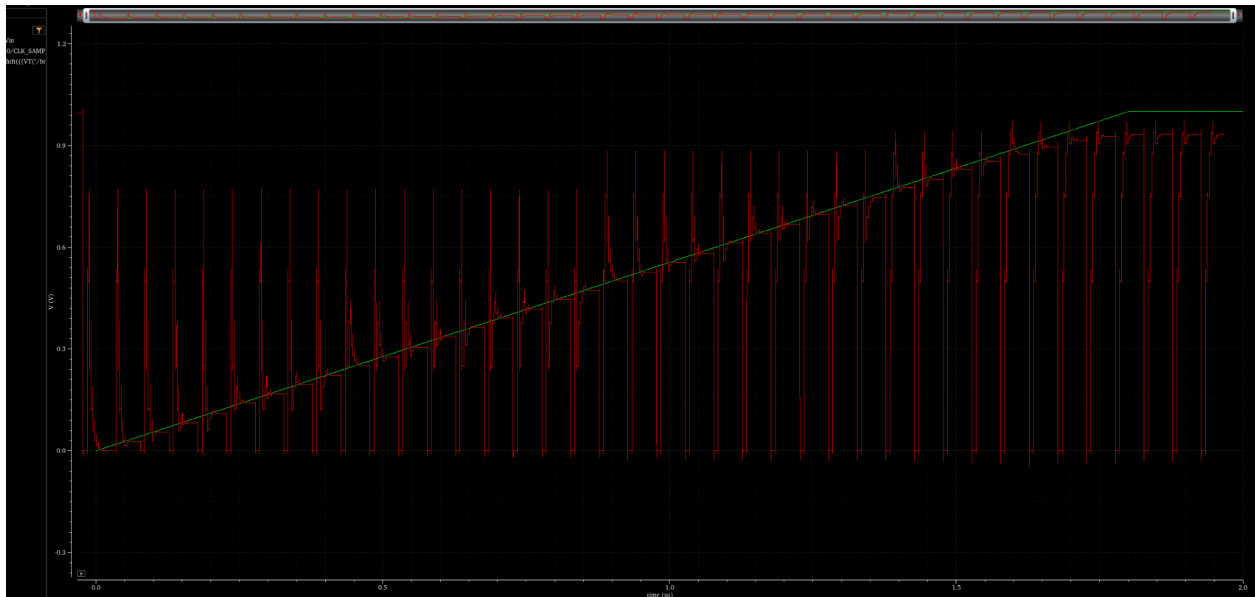
Our development process can roughly be broken down into iterating through schematic designs and simulations. We simulated our design through the use of Falstad (online circuit JS simulator) for checking the logic of our D Flip-Flop ring counter and bit register. For electrical simulation and testing of the full function of our SAR ADC we utilized the included ADE Explorer within Cadence Virtuoso, along with Viva and the built in calculation tools for analysis. Our methodology was to begin with each bottom level component, test individually for the functionality, integrate, test, then repeat until we built up a full ADC schematic. We used the simulation results to guide our development and decisions throughout the process:



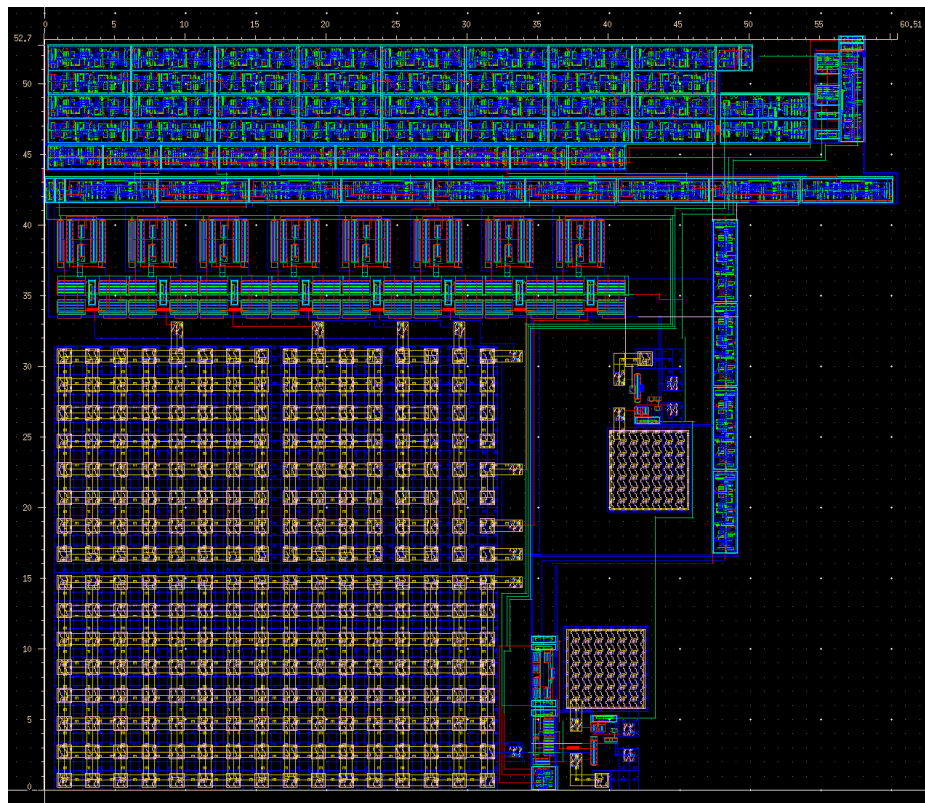
- Observing that we had bit errors at around MSB and 2ndMSB, probed it, and slowed down CLK_SAR via delays.

Below is a sweep where the input voltage is ramped from 0 to 1V, not using our extracted results. We are plotting our converted voltage with the equation “ $\text{lshift}((\text{VT}("/\text{bit_7}") * 0.5) + (\text{VT}("/\text{bit_6}") * 0.25) + (\text{VT}("/\text{bit_5}") * 0.125) + (\text{VT}("/\text{bit_4}") * 0.0625) + (\text{VT}("/\text{bit_3}") * 0.03125) + (\text{VT}("/\text{bit_2}") * 0.015625) + (\text{VT}("/\text{bit_1}") * 0.0078125) + (\text{VT}("/\text{bit_0}") * 0.00390625)) 33n$ ” (shifted to align the decision with the start of sampling to compare more easily)

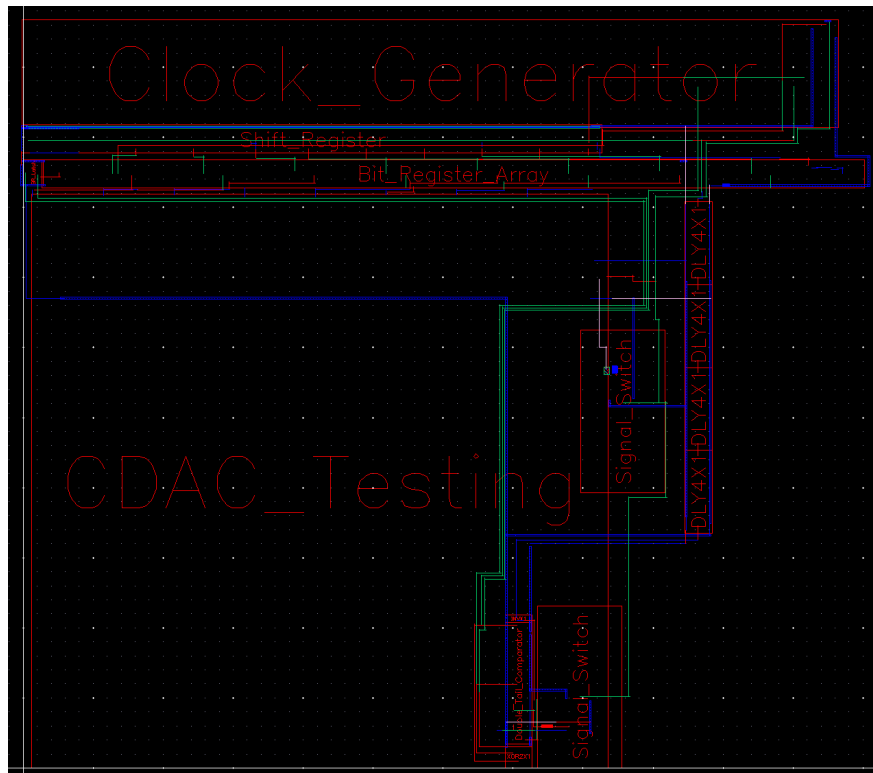
Future work would involve more fine grain steps and analysis at each bit transition.



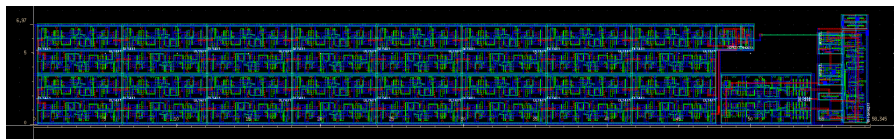
Layout and Extraction Results



Entire Layout of the SAR ADC



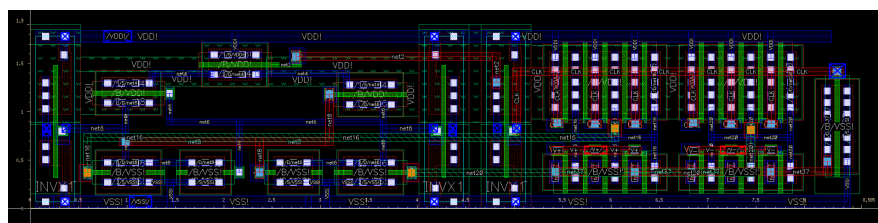
Block Model of the Entire Layout



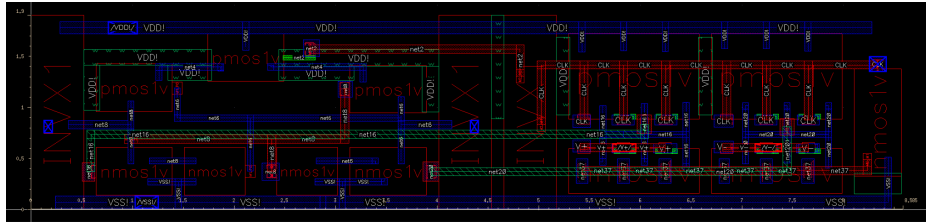
Clock Generator Circuit Layout



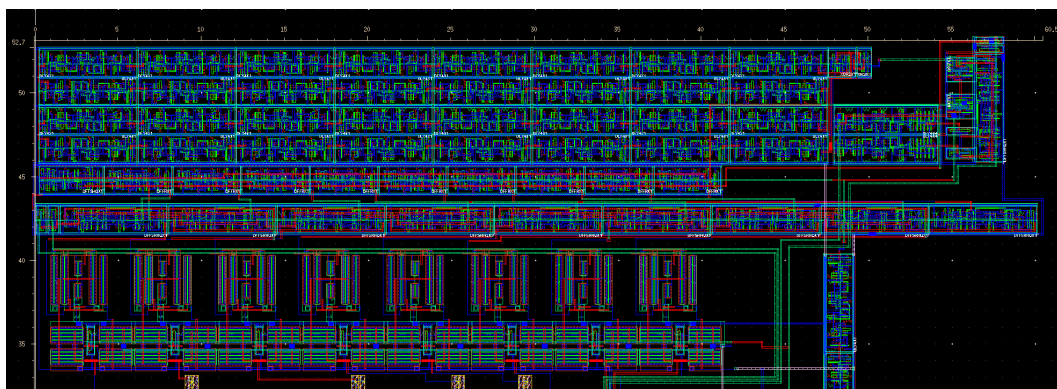
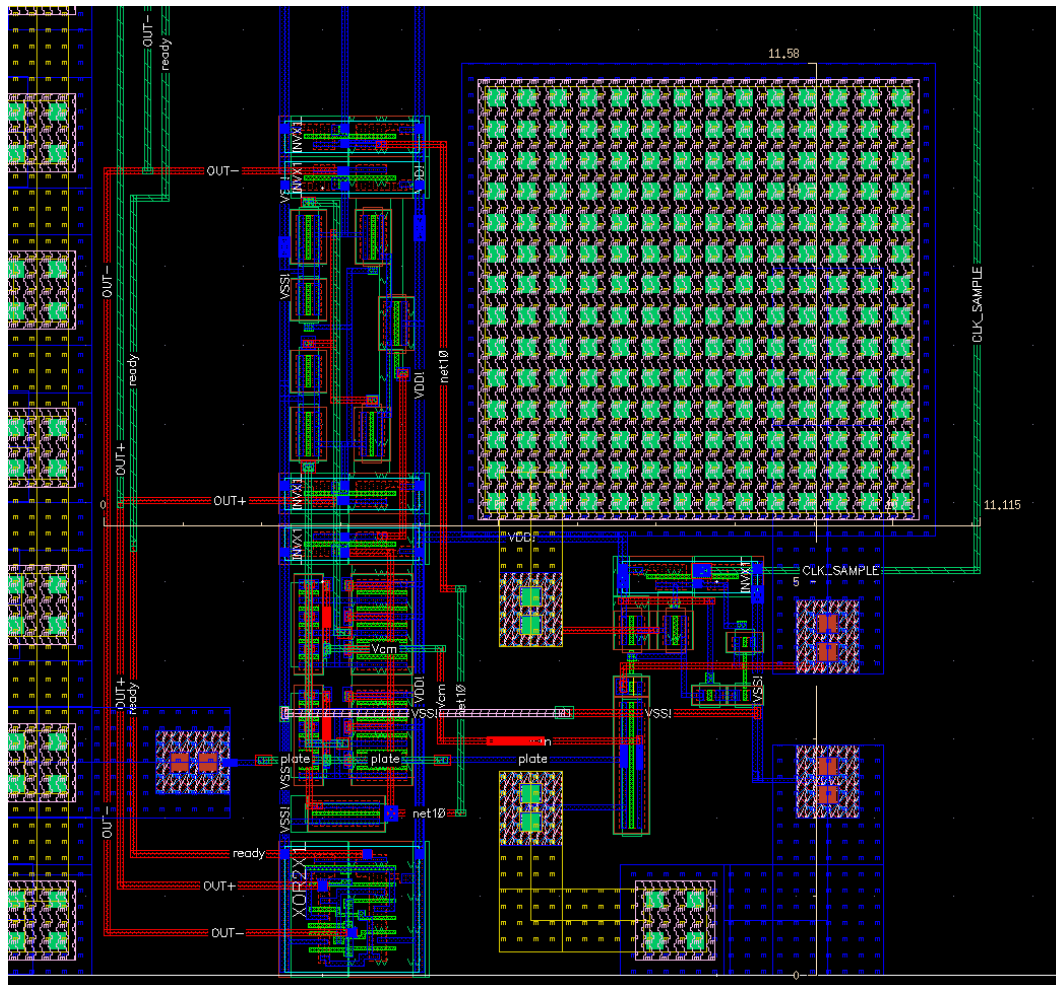
Clock Generator Block Model Layout



Layout of the Double-Tail Comparator



Block Model of the Double-Tail Comparator Layout



Below is our config for a test bench where we have set the SAR_ADC to the extracted view.

Virtuoso® Hierarchy Editor Editing: (SAR_ADC Sim_SAR_ADC_Final_DBG config) x

Launch File Edit View Help cadence

Invalid Cells: 1

Top Cell ? [icon] x

Library: SAR_ADC

Cell: Sim_SAR_ADC_Final_DBG

View: schematic

Open Edit ADE Explorer

Global Bindings ? [icon] x

Library List: tre cmos_sch schematic ahdl ...

View List: spectre ...

Stop List: ...

Constraint List: ...

Table View Tree View

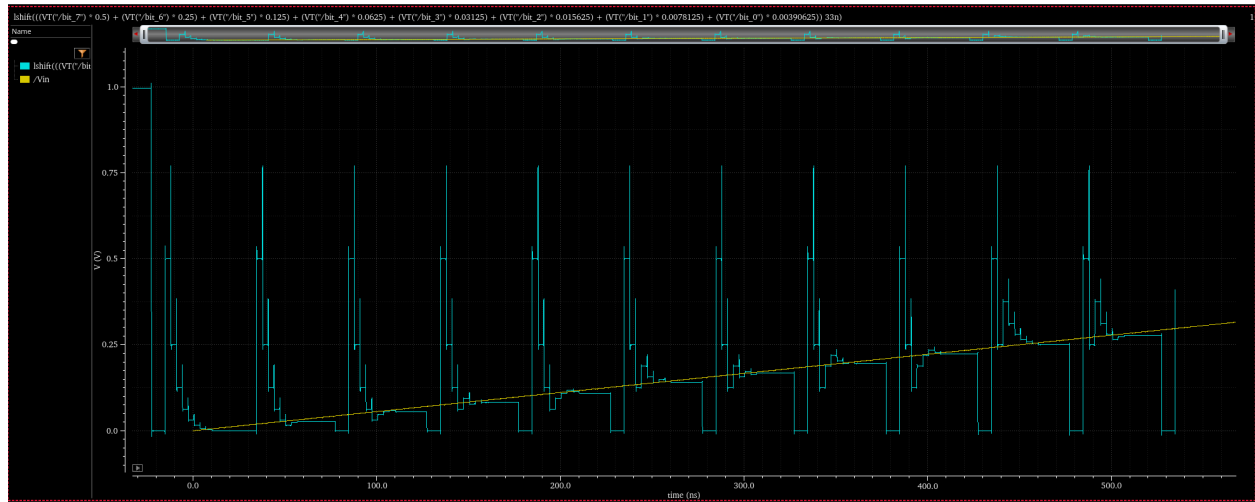
Cell Bindings

	Library	Cell	View Found	View To Use	Inherited View List
i	SAR_ADC	SAR_ADC	av_extracted	av_extracted	spectre
[icon]	SAR_ADC	Sim_SAR_ADC_Final_...	schematic		spectre
	analogLib	pcapacitor	spectre		spectre
	analogLib	presistor	spectre		spectre
	analogLib	vdc	spectre		spectre
	analogLib	vpulse	spectre		spectre
	analogLib	vsource	spectre		spectre
	gpd045	mimcap	spectre		spectre
	gpd045	nmos1v	spectre		spectre
	gpd045	pmos1v	spectre		spectre
[icon]	gsclib045	INVX1	**NONE**		spectre

Namespace: CDBA Filters: OFF

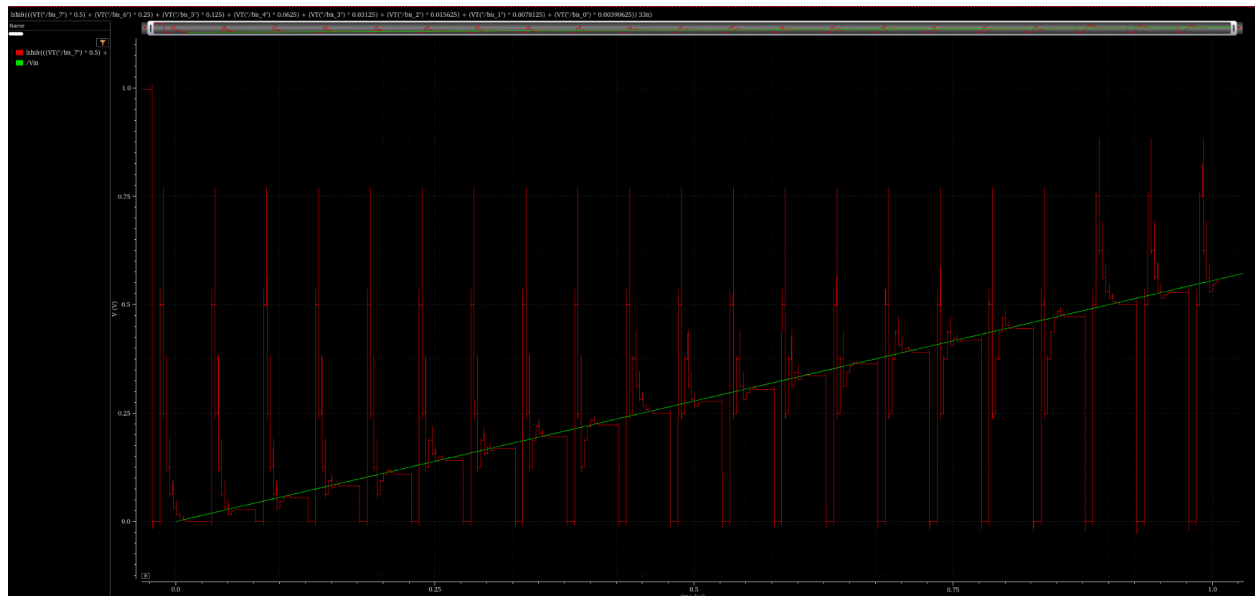
14(35) >

Running with our extracted design, we noticed a number of circuit functionality changes, some of which took time to redesign and fix. Most noticeably, we began seeing very strong kickback from our comparator when crossing boundaries where the MSB or second MSB flipped.



In order to fix these issues, we added delay elements into our feedback loop to prevent our comparator from evaluating a still fluctuating value. Our CDAC saw large voltage swings when charging and discharging, particularly when transitioning from evaluating an analog value with MSB = 0 to MSB = 1. By adding the delay, we gave the capacitor bank time to settle to the appropriate value before starting the evaluation process.

Here is a final graph of our extracted layout's ADC measurement vs an input sweep. Future work would involve more comprehensive bit evaluation.



Conclusions

Through our work on this project we gained insight into the difficulties of analog circuit design. Primarily, we were concerned with the effects of noise and the challenges associated with properly sizing our transistors. Compared to our midterm goals we ended up implementing multiple features. We believe that there is further room to improve the design as we underestimated just how complicated even an architecture as straightforward as a SAR ADC would be to implement in analog.

Further improvements would include higher accuracy and linearity. While our ADC is able to read many values properly, it has particular difficulty when it reaches the upper end of its measurement range. We would want to dedicate more time to fixing this issue to provide a more linear and accurate design.

To reference our design and testbench, view our files at the following path:
`/home/xpa7059/ce393/SAR_ADC`